

Dig Dug Repair Lab

This project is a single-file Dig Dug-lite clone using **Pygame**. It introduces students to grid digging, breadth-first pathfinding, and multi-stage enemy state using a small, readable object-oriented codebase.

What's Provided

A working Dig Dug-lite game with:

- A player that digs through dirt as they move, leaving permanent tunnels behind them
- Enemies that path toward the player through cleared tunnels using breadth-first search, and can turn "ghost" to phase through dirt briefly
- A pump attack that inflates an enemy through four stages before popping it
- Levels, lives, and scoring

It has **one deliberate bug** and **three optional features** left as empty functions. You are expected to **analyze, interact with an AI assistant, and complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
pip install pygame
```

3. Run the game:

```
python game.py
```

Controls: Arrow keys to dig/move, Space to pump, R to reset.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the enemy pathfinding bug

Once a tunnel connects an enemy to the player, the enemy is supposed to take the *shortest* route through cleared cells, using breadth-first search. In the current build, enemies do eventually reach the player, but along long, winding routes instead of the shortest one — the search behaves like depth-first search rather than breadth-first. Look at which end of the `deque` the search removes cells from in `bfs_path`.

Task 2: Implement `dirt_color(row)`

Called once per dirt cell per frame in `draw`, as `shade = dirt_color(row)` or `(170 - row * 6, 110 - row * 4, 60)`. It receives the cell's row index and should return an `(r, g, b)` color, or `None` to keep the default gradient (which darkens with depth). Idea: give each row band a distinct color instead of a smooth gradient.

Task 3: Implement `on_enemy_popped(enemy, score)`

Called from `pump()` right after an enemy reaches its fourth inflation stage, is removed from play, and its points are added. It receives the `Enemy` that popped and the score after its points were added. Its return value is ignored. Idea: a particle burst at `enemy.cell`, or extra points for popping an enemy deep underground.

Task 4: Implement `enemy_speed_multiplier(level)`

Called every time a non-locked enemy finishes a step, as part of `enemy.step_timer = ENEMY_DELAY * (1.6 if enemy.ghost else 1) / (enemy_speed_multiplier(self.level) or 1)`. It receives the current level number and should return a speed

multiplier, or `None` for the default speed (1x). A higher multiplier makes enemies move more often (shorter delay between steps). Idea: return $1 + 0.1 * \text{level}$ so enemies get gradually faster on deeper levels.

Expected Behavior

- Moving the player clears dirt in the cells they move into
 - A pumped enemy can't move, and slowly deflates back down if you stop pumping it before it pops
 - Enemies path through cleared tunnels toward the player using short routes; while "ghosting" they move in straight lines through dirt instead
 - Popping every enemy on a level advances to the next, larger level
 - Touching a non-locked enemy costs a life; the game ends when lives reach zero
-

Folder Structure

```
dig_dug/  
├── game.py  
└── README.md
```

Submission Checklist

Submission is only the following three things:

- ☐ A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- ☐ A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- ☐ The Chat/LLM used page link, with the complete chat history
- give the task 1 i need to do

Siddu, let's complete **Task 1: Fix the enemy pathfinding bug** step by step.

1. Understand the bug

Your game uses **Breadth-First Search (BFS)** to find the shortest path from an enemy to the player through cleared tunnels.

The problem is likely in the `bfs_path()` function.

BFS uses a queue (`deque`) and follows **FIFO (First In, First Out)**.

- BFS: removes the oldest element first.
- DFS: typically explores the newest element first.

If the code uses `queue.pop()` instead of `queue.popleft()` , it behaves like a stack and explores paths in a DFS-like manner.

2. Find the function in `game.py`

Look for something similar to this:

`</>` Python

```
from collections import deque

def bfs_path(start, goal, grid):
    queue = deque([start])
    visited = {start}
    parent = {}

    while queue:
        current = queue.pop() # BUG: behaves Like DFS

        if current == goal:
            break

        for neighbor in get_neighbors(current, grid):
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = current
                queue.append(neighbor)

    # Reconstruct path
```

This is an illustrative version. Your actual function might have a different signature or implementation.

3. Fix the bug

Replace:

</> Python

```
current = queue.pop()
```

with:

</> Python

```
current = queue.popleft()
```

That's the main fix.

Why does this work?

Before: DFS-like

Last element is removed first.

</> Python

```
queue.pop()
```

After: BFS

First element is removed first.

</> Python

```
queue.popleft()
```

4. Test your fix

1. Run your game:

</> Bash

```
python game.py
```

2. Move around to create a network of tunnels.
3. Allow an enemy to enter the cleared tunnel network.
4. Observe whether the enemy follows a shorter route toward you rather than taking a winding route.

For a more reliable test, you can also test BFS independently with a small grid and compare the resulting path length with the expected shortest distance.

5. What to write in your lab report

Writing



Task 1: Fix the enemy pathfinding bug

Problem: The enemy pathfinding algorithm was behaving like Depth-First Search (DFS) instead of Breadth-First Search (BFS), resulting in unnecessarily long paths.

Cause: The `bfs_path()` function used `deque.pop()`, which removes elements from the right end of the deque, causing LIFO-like exploration.

Solution: Replaced `queue.pop()` with `queue.popleft()`. This makes the search follow FIFO order, as required by BFS.

Result: Enemies can find shortest paths through cleared tunnels, assuming all moves have equal cost and the BFS implementation correctly handles visited cells and path reconstruction.

Now, Siddu: Open your actual `game.py` and paste the `bfs_path()` function here. I'll help you identify the exact line to change so you can fix Task 1 in your own project without accidentally modifying other code.

give teh task2?

Siddu, let's complete **Task 2: Implement** `dirt_color(row)` in your Dig Dug game.

1. Understand the task

Currently, the game uses this code in its `draw()` function:

`</>` Python

```
shade = dirt_color(row) or (170 - row * 6, 110 - row * 4, 60)
```

Here:

- `dirt_color(row)` receives the row number of a dirt cell.
- It should return an RGB color tuple.
- If it returns `None`, the game uses the default gradient.
- We want to give different underground layers distinct colors instead of using a smooth gradient.

2. Implement `dirt_color(row)`

Replace the empty function with this:

Writing



```
def dirt_color(row):
    if row < 3:
        return (194, 157, 105) # Light sandy soil
    elif row < 6:
        return (160, 110, 65) # Brown soil
    elif row < 9:
        return (120, 75, 45) # Dark brown soil
    else:
        return (85, 50, 35) # Deep underground soil
```

3. Understand the code

Condition	RGB color	Layer
<code>row < 3</code>	<code>(194, 157, 105)</code>	Light sandy soil
<code>row < 6</code>	<code>(160, 110, 65)</code>	Brown soil
<code>row < 9</code>	<code>(120, 75, 45)</code>	Dark brown soil
Otherwise	<code>(85, 50, 35)</code>	Deep underground soil

The RGB format represents red, green and blue values, each ranging from 0 to 255.

As the row number increases, the soil gets darker, making the underground layers visually distinct.

4. Test your implementation

Run your game:

`</>` Bash

```
python game.py
```

Check the following:

- The upper dirt rows have a lighter sandy color.
- The middle rows have brown shades.
- The lower rows have darker shades.
- The player can still dig through all the layers normally.

If your game has fewer or more rows, adjust the row thresholds accordingly.

5. What to write in your lab report

Writing



Task 2: Implement dirt_color(row)

****Problem:**** The original game used a smooth gradient for dirt colors, making underground layers visually uniform.

****Solution:**** Implemented the `dirt\_color(row)` function to return different RGB colors based on the row index.

****Implementation:**** Used conditional statements to divide the underground into four color bands: light sandy soil, brown soil, dark brown soil and deep underground soil.

****Result:**** The dirt now has visually distinct underground layers, improving the appearance of the game while preserving the existing digging mechanics.

Important: This implementation assumes your game has at least 10 rows. If your existing game has a different grid size, you can adjust the thresholds to suit it.

Next, you can implement Task 3, where popping an enemy triggers a particle effect or awards bonus points.

[give the task 3?](#)

Siddu, let's complete **Task 3: Implement** `on_enemy_popped(enemy, score)` in your Dig Dug game.

For this task, let's add a **bonus scoring system**: enemies popped deeper underground give extra points. This is simple to implement and easy to demonstrate in your gameplay video.

1. Understand the task

The function is called when an enemy reaches its fourth inflation stage and is popped.

</> Python

```
def on_enemy_popped(enemy, score):  
    pass
```

- `enemy` : the enemy object that was popped.
- `score` : the player's score after the enemy's points have already been added.
- Return value: ignored.

We can use the enemy's row position to award bonus points for deeper enemies.

2. Implement `on_enemy_popped()`

Replace the empty function with:

Writing



```
def on_enemy_popped(enemy, score):  
    row = enemy.cell[0]  
  
    bonus = row * 50  
  
    print(f"Enemy popped at depth {row}!")  
    print(f"Bonus points: {bonus}")  
    print(f"Current score: {score + bonus}")
```

How it works:

- `enemy.cell[0]` gets the row coordinate of the enemy.
- `row * 50` calculates the bonus points.
- Deeper enemies earn larger bonuses.
- The `print()` statements display the bonus and updated score in the terminal.

Important: This implementation only displays the bonus. It does not actually add it to the game's score because the callback's return value is ignored and the existing score is managed elsewhere.

If you want the bonus to count in the game, you need to modify the scoring logic in `pump()` or another part of your game.

3. Optional: Add a particle burst

If you want a visual effect instead, you can create a simple particle burst. However, this requires additional game state and drawing logic, so the bonus display is a simpler first implementation.

4. Test your implementation

Run:

`</> Bash`

```
python game.py
```

Then:

1. Pump an enemy until it pops.
2. Observe the terminal output.
3. Check the enemy's depth and the calculated bonus.
4. Pop enemies at different depths to compare the bonuses.

5. Lab report

Writing



Task 3: Implement on_enemy_popped(enemy, score)

Problem: The original callback function was empty and did not provide any additional feedback when an enemy was popped.

Solution: Implemented the callback to calculate bonus points based on the enemy's depth.

Implementation: Used the enemy's row coordinate to calculate a bonus of 50 points per row. Added terminal output to display the enemy's depth, bonus points and projected score.

Result: The callback now provides depth-based bonus information whenever an enemy is popped. The bonus is displayed but is not added to the actual game score.

Note: If your lab evaluator expects the bonus to appear in the actual game score or a particle burst to be visible in the gameplay video, this version alone will not satisfy that requirement. It is a functional callback demonstration, not a complete in-game feature.

give teh task 4?

Yes. **Task 4 is the easiest one.** You need to implement `enemy_speed_multiplier(level)` so enemies gradually become faster on higher levels.

Task 4 — `enemy_speed_multiplier(level)`

Find the empty function in `game.py`, probably something like:

```
</> Python
```

```
def enemy_speed_multiplier(level):  
    pass
```

Replace it with:

```
</> Python
```

```
def enemy_speed_multiplier(level):  
    return 1 + 0.1 * level
```

How it works

The game already uses:

```
</> Python
```

```
enemy.step_timer = ENEMY_DELAY * (1.6 if enemy.ghost else 1) / (enemy_speed_multiplier(self.level) or 1)
```

So:

Level	Multiplier	Enemy speed
1	1.1	Slightly faster
2	1.2	Faster
3	1.3	Faster
5	1.5	Much faster
10	2.0	Twice as fast

Because the delay is **divided** by the multiplier, a higher multiplier means a **shorter delay** between enemy movements.

For example, if:

</> Python

```
ENEMY_DELAY = 10
```

At level 1:

</> Plain text

```
10 / 1.1 ≈ 9.09
```

At level 5:

</> Plain text

```
10 / 1.5 ≈ 6.67
```

So enemies move more frequently as the level increases.

Test it

Run:

</> Bash

```
python game.py
```

Then play through multiple levels. You should notice that enemies move progressively faster.

For your lab report

Task 4 — Enemy Speed Multiplier



Task 4: Implement `enemy_speed_multiplier(level)`

Problem: The `enemy_speed_multiplier()` function was empty, so enemy speed remained unchanged across different levels.

Solution: Implemented the function using `1 + 0.1 * level`.

Implementation:

```
def enemy_speed_multiplier(level):  
    return 1 + 0.1 * level
```

Result: The enemy speed now increases gradually with the level. Higher levels produce a larger speed multiplier, reducing the delay between enemy movements and making later levels progressively more challenging.

Your 4 tasks are now

1. Task 1: `queue.pop()` → `queue.popleft()` 
2. Task 2: Implement `dirt_color(row)` 
3. Task 3: Implement `on_enemy_popped(enemy, score)` 
4. Task 4: Implement `enemy_speed_multiplier(level)` 

For your **before/after 10-second videos**, make sure the **before video** shows the original broken behavior and the **after video** shows the pathfinding fix + visible effects/features from Tasks 2–4.